N-Gram Language Models

Generating Text Without Neural Networks!

CSE 534

Goal: Build a simple text generator using just probability and counting
No training GPUs needed - just count character frequencies

What is an N-Gram?

N-gram: Sequence of N consecutive items (characters, words, tokens)

Examples with "hello␣world":

- **1-gram (unigram):** `h`, `e`, `l`, `l`, `o`, `␣`, `w`, `o`, `r`, `l`, `d`
- **2-gram (bigram):** `he`, `el`, `ll`, `lo`, `o␣`, `␣w`, `wo`, `or`, `rl`, `ld`
- **3-gram (trigram):** `hel`, `ell`, `llo`, `lo␣`, `o␣w`, `␣wo`, `wor`, `orl`, `rld`

N-Grams for Text Generation

Idea: Use previous N-1 characters to predict next character

N-grams from a large corpus (including "hello"):

n-gram	count
^^h	30
^he	10
hel	5
...	...

Current state:

```
Generated: ^^
```

Generation Step 1

Trigrams starting with "^^":

n-gram	count
^^h	30
^^w	10
^^j	10
...	...

Total: 50

Generated so far: ^^

Current context: ^^

Probabilities:

Selected: h

Generation Step 2

Trigrams starting with "^h":

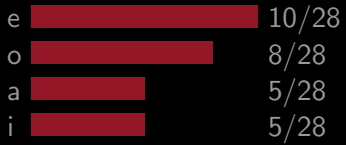
n-gram	count
^he	10
^ho	8
^ha	5
^hi	5
...	...

Total: 28

Generated so far: `^^h`

Current context: `^h`

Probabilities:



Selected: e

Generation Step 3

Trigrams starting with “he”:

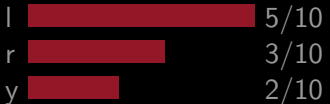
n-gram	count
hel	5
her	3
hey	2
...	...

Total: 10

Generated so far: `^^he`

Current context: `he`

Probabilities:



Selected: l

Generation Step 4

Trigrams starting with “el”:

n-gram	count
ell	25
els	8
ele	5
...	...

Total: 38

Generated so far: `^^hel`

Current context: `el`

Probabilities:

Selected: l

Generation Step 5

Trigrams starting with “ll”:

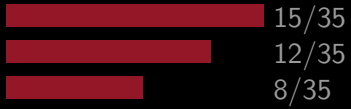
n-gram	count
llo	15
lly	12
lle	8
...	...

Total: 35

Generated so far: `^^hell`

Current context: `ll`

Probabilities:



Selected: o

Final result: `^^hello`

Why Does This Work?

We just generated text by counting! But why does this actually work?

The Theory:

- **Autoregressive:** Output at each step depends on the model's own previous outputs

Why Does This Work?

We just generated text by counting! But why does this actually work?

The Theory:

- **Autoregressive:** Output at each step depends on the model's own previous outputs
- Every generative model needs $P(\text{next} \mid \text{context})$

Why Does This Work?

We just generated text by counting! But why does this actually work?

The Theory:

- **Autoregressive:** Output at each step depends on the model's own previous outputs
- Every generative model needs $P(\text{next} \mid \text{context})$
- Understanding N-gram math = understanding **all** language models

Why Does This Work?

We just generated text by counting! But why does this actually work?

The Theory:

- **Autoregressive:** Output at each step depends on the model's own previous outputs
- Every generative model needs $P(\text{next} \mid \text{context})$
- Understanding N-gram math = understanding **all** language models
- Same probability foundations power both simple and complex models

Probability: Joint vs Conditional

Let's put this into the context of generative modeling.
We are learning a model for $p(\text{sequence})$.

Goal: Model probability of entire sequence

Example: $p(\text{hello}) = p(h, e, l, l, o)$

Chain Rule - Step by Step

Chain rule factorizes joint into conditionals:

$$p(\text{hello}) = \underbrace{p(h)}_{\text{easy!}} \cdot p(\text{ello}|h)$$

Chain Rule - Step by Step

Chain rule factorizes joint into conditionals:

$$p(\text{hello}) = \underbrace{p(h)}_{\text{easy!}} \cdot p(\text{ello}|h)$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(\text{llo}|\text{he})$$

Chain Rule - Step by Step

Chain rule factorizes joint into conditionals:

$$p(\text{hello}) = \underbrace{p(h)}_{\text{easy!}} \cdot p(\text{ello}|h)$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(\text{llo}|\text{he})$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(l|\text{he}) \cdot p(\text{lo}|\text{hel})$$

Chain Rule - Step by Step

Chain rule factorizes joint into conditionals:

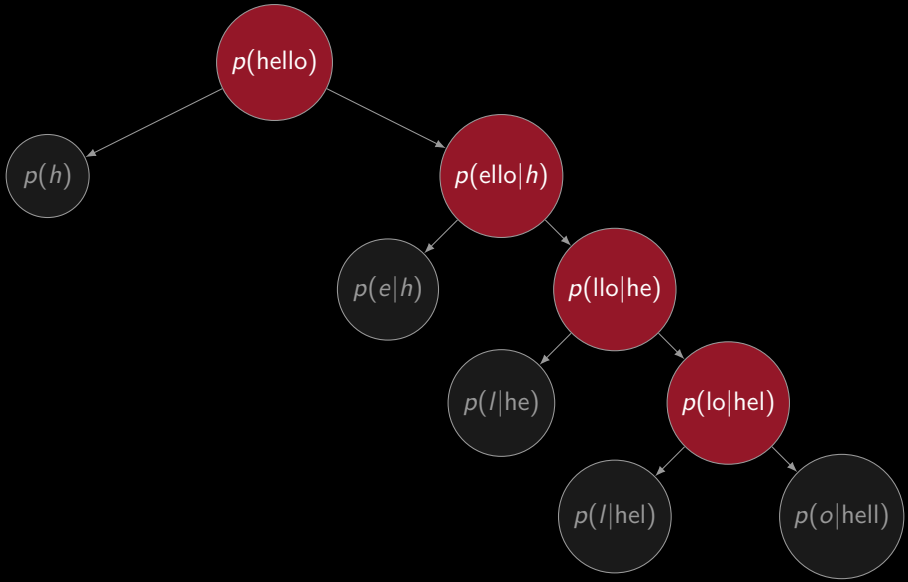
$$p(\text{hello}) = \underbrace{p(h)}_{\text{easy!}} \cdot p(\text{ello}|h)$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(\text{llo}|\text{he})$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(l|\text{he}) \cdot p(\text{lo}|\text{hel})$$

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(l|\text{he}) \cdot p(l|\text{hel}) \cdot p(o|\text{hell})$$

Chain Rule Visualization



General Pattern

$$p(x_1, \ldots, x_n) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdots p(x_n|x_1, \ldots, x_{n-1})$$

Problem: $p(x_n|x_1, \ldots, x_{n-1})$ needs **entire history**

- For long sequences: too many possible histories!

General Pattern

$$p(x_1, \ldots, x_n) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdots p(x_n|x_1, \ldots, x_{n-1})$$

Problem: $p(x_n|x_1, \ldots, x_{n-1})$ needs **entire history**

- For long sequences: too many possible histories!
- As context gets longer, harder to find enough matches in training data

Markov Assumption

Key idea: Assume each character depends only on **last k characters**

$$p(x_n|x_1, \ldots, x_{n-1}) \approx p(x_n|x_{n-k}, \ldots, x_{n-1})$$

Markov Assumption (continued)

Order-k Markov assumption:

- Ignore history beyond k positions

Markov Assumption (continued)

Order-k Markov assumption:

- Ignore history beyond k positions
- Only use k previous characters as context

Markov Assumption (continued)

Order-k Markov assumption:

- Ignore history beyond k positions
- Only use k previous characters as context

Example with k=2:

$$p(\text{hello}) = p(h) \cdot p(e|h) \cdot p(l|\text{he}) \cdot \underbrace{p(l|\text{el})}_{\text{no 'h'!}} \cdot \underbrace{p(o|\text{ll})}_{\text{no 'he'!}}$$

“All Models Are Wrong, But Some Are Useful”

George Box (Statistician)

Every model makes simplifying assumptions

Our Markov assumption:

- Only last k characters matter

> *“All models are wrong,*
> *but some are useful”*
>
> — George Box

“All Models Are Wrong, But Some Are Useful”

George Box (Statistician)

Every model makes simplifying assumptions

Our Markov assumption:

- Only last k characters matter
- Ignore all earlier context

> *“All models are wrong,*
> *but some are useful”*
>
> — George Box

“All Models Are Wrong, But Some Are Useful”

George Box (Statistician)

Every model makes simplifying assumptions

Our Markov assumption:

- Only last k characters matter
- Ignore all earlier context
- This is **wrong**... but **useful**!

> *“All models are wrong, but some are useful”*
>
> — George Box

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry
- **Hierarchical structure:** Nested parentheses or balanced brackets

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry
- **Hierarchical structure:** Nested parentheses or balanced brackets
- **Pattern repetition:** Train on "bonbon" $\rightarrow$ "bonbonbonbon..." forever!

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry
- **Hierarchical structure:** Nested parentheses or balanced brackets
- **Pattern repetition:** Train on "bonbon" $\rightarrow$ "bonbonbonbon..." forever!
- **Sparse contexts:** ``To be or'' appears rarely in Shakespeare

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry
- **Hierarchical structure:** Nested parentheses or balanced brackets
- **Pattern repetition:** Train on "bonbon" $\rightarrow$ "bonbonbonbon..." forever!
- **Sparse contexts:** ``To be or'' appears rarely in Shakespeare
- **Dead ends:** Typo or rare prefix $\rightarrow$ no continuation $\rightarrow$ STOP

Inductive Bias and Limitations

Fixed context window = structural constraint (inductive bias)

The model cannot learn:

- **Long-range dependencies:** Matching quotes 100 chars apart
- **Palindromes:** No understanding of symmetry
- **Hierarchical structure:** Nested parentheses or balanced brackets
- **Pattern repetition:** Train on "bonbon" $\rightarrow$ "bonbonbonbon..." forever!
- **Sparse contexts:** ``To be or'' appears rarely in Shakespeare
- **Dead ends:** Typo or rare prefix $\rightarrow$ no continuation $\rightarrow$ STOP

Trade-off: Larger k needs exponentially more data (vocab^k contexts)

Modern transformers use learned attention to relax these constraints

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian idea: Combine prior belief with observed data

$$p(\mu|D) = \frac{p(D|\mu) \cdot p(\mu)}{p(D)}$$

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian idea: Combine prior belief with observed data

$$p(\mu|D) = \frac{p(D|\mu) \cdot p(\mu)}{p(D)}$$

- $p(\mu)$ = **prior**: belief before seeing data

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian idea: Combine prior belief with observed data

$$p(\mu|D) = \frac{p(D|\mu) \cdot p(\mu)}{p(D)}$$

- $p(\mu)$ = **prior**: belief before seeing data
- $p(D|\mu)$ = **likelihood**: how well data fits model

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian idea: Combine prior belief with observed data

$$p(\mu|D) = \frac{p(D|\mu) \cdot p(\mu)}{p(D)}$$

- $p(\mu)$ = **prior**: belief before seeing data
- $p(D|\mu)$ = **likelihood**: how well data fits model
- $p(\mu|D)$ = **posterior**: updated belief after seeing data

Bayesian Smoothing: Handling Unseen Contexts

Problem: What if we encounter a context we never saw in training?

Without smoothing: $p(x|c) = 0$ for unseen pairs $\rightarrow$ `STOP` or infinite loss

Bayesian idea: Combine prior belief with observed data

$$p(\mu|D) = \frac{p(D|\mu) \cdot p(\mu)}{p(D)}$$

- $p(\mu)$ = **prior**: belief before seeing data
- $p(D|\mu)$ = **likelihood**: how well data fits model
- $p(\mu|D)$ = **posterior**: updated belief after seeing data

(Full conjugate prior theory is for advanced courses)

Prior as Imaginary Experiments

Intuition: Pretend we did α "prior experiments" with uniform results

Smoothing formula:

$$\hat{p}(x|c) = \frac{\text{count}(c, x) + \alpha}{\text{count}(c) + \alpha \cdot |\text{vocab}|}$$

Prior as Imaginary Experiments

Intuition: Pretend we did α "prior experiments" with uniform results

Smoothing formula:

$$\hat{p}(x|c) = \frac{\text{count}(c, x) + \alpha}{\text{count}(c) + \alpha \cdot |\text{vocab}|}$$

Interpretation:

- Imagine we did α experiments before training

Prior as Imaginary Experiments

Intuition: Pretend we did α "prior experiments" with uniform results

Smoothing formula:

$$\hat{p}(x|c) = \frac{\text{count}(c, x) + \alpha}{\text{count}(c) + \alpha \cdot |\text{vocab}|}$$

Interpretation:

- Imagine we did α experiments before training
- Each experiment produced a uniform distribution over all characters

Prior as Imaginary Experiments

Intuition: Pretend we did α "prior experiments" with uniform results

Smoothing formula:

$$\hat{p}(x|c) = \frac{\text{count}(c, x) + \alpha}{\text{count}(c) + \alpha \cdot |\text{vocab}|}$$

Interpretation:

- Imagine we did α experiments before training
- Each experiment produced a uniform distribution over all characters
- α doesn't have to be an integer! We can use $\alpha = 0.01$

Prior as Imaginary Experiments

Intuition: Pretend we did α "prior experiments" with uniform results

Smoothing formula:

$$\hat{p}(x|c) = \frac{\text{count}(c, x) + \alpha}{\text{count}(c) + \alpha \cdot |\text{vocab}|}$$

Interpretation:

- Imagine we did α experiments before training
- Each experiment produced a uniform distribution over all characters
- α doesn't have to be an integer! We can use $\alpha = 0.01$
- This adds α "virtual counts" for each character

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Example with vocab size 100:

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Example with vocab size 100:

- $\alpha = 1$: Equivalent to 100 prior observations (too strong!)

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Example with vocab size 100:

- $\alpha = 1$: Equivalent to 100 prior observations (too strong!)
- $\alpha = 0.01$: Equivalent to 1 prior observation total

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Example with vocab size 100:

- $\alpha = 1$: Equivalent to 100 prior observations (too strong!)
- $\alpha = 0.01$: Equivalent to 1 prior observation total
- Prevents division by zero without overwhelming real data

Choosing α: How Much to Trust the Prior

How much weight should we give to the prior?

- $\alpha = 1$: Add-one (Laplace) smoothing—1 uniform experiment per character
- $\alpha = 0.01$: Weak prior—0.01 uniform experiments per character
- Smaller α = trust real data more, imaginary prior less

Example with vocab size 100:

- $\alpha = 1$: Equivalent to 100 prior observations (too strong!)
- $\alpha = 0.01$: Equivalent to 1 prior observation total
- Prevents division by zero without overwhelming real data

Rule of thumb: $\alpha \ll 1$ prevents gibberish while trusting your corpus

Training: Count N-Grams

Step 1: Fetch corpus (Shakespeare plays)

```
from urllib.request import urlopen

url = "https://raw.githubusercontent.com/karpathy/\
char-rnn/master/data/tinyshakespeare/input.txt"
with urlopen(url) as r:
    corpus = r.read().decode('utf-8')
```

Step 2: Define parameters

```
ORDER = 8    # context length (8-char context -> 9-gram)
PAD = "^"    # padding at start
END = "$"    # end marker
```

Training: Counting Loop

Step 3: Count frequencies

```
count = {}  # count[context] = {next_char: frequency}

for block in blocks:
    seq = PAD * ORDER + block + END  # pad start, mark end

    for i in range(len(seq) - ORDER):
        ctx = seq[i:i+ORDER]           # 8 characters
        next_char = seq[i+ORDER]        # 1 character

        if ctx not in count:
            count[ctx] = {}
        count[ctx][next_char] = \
            count[ctx].get(next_char, 0) + 1
```

Training: Save Model

Step 4: Save counts

```
import json

with open("ngram_counts.json", "w") as f:
    json.dump(count, f)
```

Result: Dictionary mapping contexts to character frequencies

```
{
  "^^^^^^^^": {"T": 100, "A": 50, ...},
  "^^^^^^^T": {"o": 80, "h": 20, ...},
  ...
  "To be or": {"n": 42, " ": 15, "d": 3},
  "be or no": {"t": 87},
  ...
}
```

Key insight: We've estimated $p(\text{next\_char} \mid \text{context})$ by **counting!**

Prediction: Load and Initialize

Step 1: Load counts

```
with open("ngram_counts.json") as f:
    count = json.load(f)
```

Step 2: Start with padding

```
import numpy as np

rng = np.random.default_rng()
gen = PAD * ORDER  # "^^^^^^^^^"
```

Prediction: Generation Loop

Step 3: Generate one character at a time

```
for _ in range(MAX_LEN):
    ctx = gen[-ORDER:]              # last 8 characters
    ctx_counts = count.get(ctx, {}) # get observed frequencies

    if not ctx_counts:              # unseen context
        break

    # Convert counts to probabilities and sample
    chars = list(ctx_counts.keys())
    probs = np.array([ctx_counts[c] for c in chars])
    probs = probs / probs.sum()     # normalize

    next_char = rng.choice(chars, p=probs)
    gen += next_char

    if next_char == END:
        break
```

Prediction: Output

Step 4: Clean up output

```
# Strip padding and end marker
output = gen[ORDER:].replace(END, "")
print(output)
```

Example outputs:

```
ROMEO:
O, that I were a glove upon that hand,
That I might touch that cheek!

NURSE:
What say you? can you love the gentleman?
```

Surprisingly coherent for such a simple model!

Live Demo

Open terminal and run:

```
cd mathematical_foundations/07_ngram
python 07_ngram_train.py            # count N-grams
python 07_ngram_predict.py          # basic (may hit dead ends)
python 07_ngram_predict_smoothed.py # with Bayesian smoothing
```

Compare outputs: The smoothed version never stops early

Evaluation: Perplexity

Recall from last lecture: Perplexity measures model quality

$$\text{perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|c_t)\right)$$

Note: Using base e (natural log) for mathematical convenience; last lecture used base 2

For N-gram models:

- c_t = the $N-1$ character context at position t

Evaluation: Perplexity

Recall from last lecture: Perplexity measures model quality

$$\text{perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|c_t)\right)$$

Note: Using base e (natural log) for mathematical convenience; last lecture used base 2

For N-gram models:

- c_t = the $N-1$ character context at position t
- Lower perplexity = better predictions on held-out text

Evaluation: Perplexity

Recall from last lecture: Perplexity measures model quality

$$\text{perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|c_t)\right)$$

Note: Using base e (natural log) for mathematical convenience; last lecture used base 2

For N-gram models:

- c_t = the $N-1$ character context at position t
- Lower perplexity = better predictions on held-out text
- Compare different N values or smoothing strategies

Evaluation: Perplexity

Recall from last lecture: Perplexity measures model quality

$$\text{perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|c_t)\right)$$

Note: Using base e (natural log) for mathematical convenience; last lecture used base 2

For N-gram models:

- c_t = the $N-1$ character context at position t
- Lower perplexity = better predictions on held-out text
- Compare different N values or smoothing strategies

Trade-off: Larger N may overfit; smaller N may underfit

Extensions and Experiments

Try different sources:

- **Pride and Prejudice:** Classic prose style
  `gutenberg.org/cache/epub/1342/pg1342.txt`
- **Alice in Wonderland:** Whimsical narrative
  `gutenberg.org/cache/epub/11/pg11.txt`
- **Django source:** Python syntax patterns
  `raw.githubusercontent.com/django/django/main/django/views/generic/base.py`

Evaluate with perplexity:

- Split data into train/test sets
- Compute NLL on held-out text: $-\frac{1}{T}\sum \log p(x_t|c_t)$
- Lower perplexity = better predictions

Modify the URL in `07_ngram_train.py` and retrain!

Characters vs Tokens

Current: Character-level

- Fine-grained
- Needs less data
- Vocab $\approx$ 100 characters

Alternative: Token-level

```
import tiktoken
enc = tiktoken.get_encoding(
    "cl100k_base"
)
tokens = enc.encode(text)
# Use tokens as keys
```

- Vocab $\approx$ 50,000 tokens
- Needs more data
- Better semantic units

Varying ORDER Parameter

ORDER	Pattern Type	Use Case
2	Simple, generic	Fast generation, minimal data
8	Medium context	What we happened to use
20	Very specific, Shakespeare-like	Huge corpus needed

Exponential growth: $\text{vocab}^{\text{ORDER}}$ possible contexts

No single "right" ORDER—depends on corpus size and desired specificity

Questions?

Try it yourself:

```
cd mathematical_foundations/07_ngram
python 07_ngram_train.py
python 07_ngram_predict.py          # may hit dead ends
python 07_ngram_predict_smoothed.py # with Bayesian smoothing
```

Experiment with:

- Different text sources
- Different ORDER values
- Character vs token-level models
- Compare basic vs smoothed generation